

CALCUL HAUTE PERFORMANCE (HPC)

Guide de Révision Magistral pour l'Examen – Synthèse des Chapitres 1 à 9

Niveau : Introduction à Avancé • **Langue :** Français • **Objectif :** Maîtrise complète des concepts théoriques et pratiques

Chapitre 1 : Introduction au Calcul Haute Performance (HPC)

1. Objectifs Généraux

Le HPC (High Performance Computing) vise à résoudre des problèmes complexes (scientifiques, industriels, financiers) en minimisant le temps de calcul. L'accent est mis sur l'efficacité matérielle et logicielle pour dépasser les limites d'un ordinateur standard.

2. Concepts Clés & Définitions

- **<>FLOPS** (Floating Point Operations Per Second) : Unité de mesure de la performance brute en calcul scientifique (opérations sur nombres à virgule flottante).
- **TOP500** : Classement bi-annuel (juin et novembre) des 500 superordinateurs les plus puissants au monde. Il se base uniquement sur le benchmark **LINPACK** (résolution d'un système d'équations linéaires dense par la méthode de Gauss). Il ne répertorie que les systèmes à usage général.
- **Évolution Architecturale** : Passage historique des machines matricielles (SIMD comme le CM-2) et vectorielles aux architectures massivement parallèles (MIMD comme IBM Blue Gene/L) et enfin aux **Clusters** modernes (ordinateurs indépendants connectés par un réseau ultra-rapide, solution économique et hautement scalable).

Exemple d'Application Réelle

Une simulation météo nécessite de calculer l'évolution de la pression sur une grille de 1 milliard de points. Si chaque point requiert 1000 opérations par seconde, un ordinateur classique (10 GFLOPS) mettrait 100 secondes par pas de temps. Un système HPC à 1 TFLOPS (1000 GFLOPS) réalise cette tâche en seulement 1 seconde, rendant la prédiction viable en temps réel.

Question Type Examen

Q : Sur quelle méthode mathématique repose le classement du TOP500 et quel benchmark est utilisé ?

R : Le classement repose sur la résolution d'un système d'équations linéaires dense en utilisant la méthode d'élimination de Gauss. Le benchmark officiel utilisé est LINPACK.

Chapitre 2 : Architecture et Performance des Processeurs

1. Architecture de Von Neumann

Comprend une unité centrale (CPU contenant l'ALU, l'unité de contrôle et les registres), une mémoire principale, et des bus d'adresse et de données. Le goulot d'étranglement majeur provient du partage des bus pour les instructions et les données.

2. Lois de Performance et Pipelining

- **Loi de Moore** : Le nombre de transistors dans un circuit intégré double environ tous les deux ans.
- **Formule du Temps d'Exécution** :

$$\text{Temps} = \text{Nombre d'Instructions} \times \text{CPI} \times \text{Période de l'Horloge}$$

Où **CPI** correspond au nombre de cycles d'horloge moyen par instruction. L'inverse du CPI est l'**IPC** (Instructions Per Cycle).

- **Pipelining** : Technique permettant de superposer l'exécution des instructions en divisant le traitement en étapes indépendantes (ex: Fetch, Decode, Execute, Memory, Write-Back).

3. Les Aléas du Pipeline (Hazards)

Type d'Aléa	Description	Solution Typique
Structurel	Deux instructions nécessitent la même ressource matérielle au même moment (ex: un seul port d'accès mémoire).	Duplication du matériel (ex: caches séparés pour instructions et données).
De Données	Une instruction dépend du résultat d'une instruction précédente encore en cours d'exécution (RAW, WAR, WAW).	Bypassing (redirection des données), bulles (stalls) ou exécution Out-of-Order.
De Contrôle	Provoqué par les instructions de saut conditionnel (if, boucles) qui modifient le compteur de programme (PC).	Prédicteurs de branchement (Branch Prediction), spéculation.

4. Mécanismes Avancés

- **Superscalaire** : Capacité du processeur à lancer plusieurs instructions en parallèle au cours d'un seul cycle grâce à de multiples unités d'exécution.
- **Out-of-Order (OoO)** : Le processeur exécute les instructions dès que leurs opérandes sont disponibles, plutôt que de suivre rigidelement l'ordre du code source.
- **Vectorisation (SIMD)** : Registres larges (SSE: 128 bits, AVX/AVX2: 256 bits, AVX-512: 512 bits) permettant d'appliquer une seule instruction à plusieurs données simultanément.

Exemple Numérique : Calcul de Temps CPU

Soit un programme de 10^9 instructions s'exécutant sur un processeur cadencé à 2.5 GHz ($2.5 \times 10^9 \text{ cycles/sec}$). Si le $\text{CPI} = 2$:

$$\text{Temps} = 10^9 \times 2 \times (1 / (2.5 \times 10^9)) = 2 \times 0.4 = 0.8 \text{ seconde}.$$

Chapitre 3 : La Hiérarchie Mémoire et le "Memory Wall"

1. Le Phénomène du Memory Wall

Le "Memory Wall" désigne l'écart croissant de performance entre la vitesse des processeurs (qui a augmenté historiquement de $\sim 3.1\times$ tous les 2 ans) et la bande passante/latence de la mémoire DRAM (qui n'a progressé que de $\sim 1.4\times$ tous les 2 ans). Le processeur passe une grande partie de son temps à attendre les données de la RAM.

2. Structure de la Hiérarchie Mémoire

Pour atténuer ce problème, la mémoire est organisée en niveaux, du plus rapide/petit au plus lent/grand :

Registres (~ 1 cycle, octets) $\rightarrow$ **Cache L1** (1-4 cycles, Ko) $\rightarrow$ **Cache L2** (10-20 cycles, Ko-Mo) $\rightarrow$ **Cache L3** (30-50 cycles, Mo) $\rightarrow$ **Mémoire RAM (DRAM)** (~ 200 cycles, Go) $\rightarrow$ **Mémoire Virtuelle / Disque** (Millions de cycles, To).

3. Principes de Localité

- **Localité Temporelle** : Une donnée accédée à un instant t a de fortes chances d'être réaccédée très prochainement (ex: variables de boucles, accumulateurs).
- **Localité Spatiale** : Une donnée située juste à côté d'une donnée accédée a de fortes chances d'être accédée juste après (ex: parcours séquentiel d'un tableau unidimensionnel).

4. Mémoire Virtuelle et Page Fault

L'espace d'adressage virtuel est découpé en blocs appelés **pages**. Un **Page Fault (défaut de page)** se produit lorsque le processeur tente d'accéder à une page mémoire virtuelle qui n'est pas chargée en RAM physique. Le système doit alors la récupérer sur le disque dur, ce qui provoque une baisse catastrophique des performances HPC.

Exemple d'Intensité Mémoire

L'intensité opérationnelle est le ratio : $I = \text{ext}\{\text{Opérations (FLOPS)}\} / \text{ext}\{\text{Accès Mémoire (Bytes)}\}$.

Pour l'opération vectorielle $Y[i] = A \text{ imes } X[i] + Y[i]$ (AXPY) : on effectue 2 opérations (1 mult, 1 add) et on requiert 3 transferts mémoire (lecture de $X[i]$, lecture de $Y[i]$, écriture de $Y[i]$). Soit $3 \times 8 \text{ ext}\{\text{octets}\} = 24 \text{ ext}\{\text{octets}\}$ pour des doubles. L'intensité est de $2 / 24 = 0.083 \text{ ext}\{\text{FLOPS/octet}\}$. C'est un code typiquement limité par la mémoire (Memory-bound).

Chapitre 4 : MPI (Message Passing Interface) - Mémoire Distribuée

1. Modèle de Programmation

MPI est le standard industriel pour l'architecture à **mémoire distribuée** (chaque processus possède son propre espace mémoire privé). La communication entre nœuds s'effectue explicitement par l'envoi et la réception de messages à travers un réseau.

2. Fonctions de Base (Initialisation et Identification)

```
#include "mpi.h"
int MPI_Init(int *argc, char ***argv); // Initialise l'environnement MPI
int MPI_Comm_size(MPI_Comm comm, int *size); // Nombre total de processus
int MPI_Comm_rank(MPI_Comm comm, int *rank); // Identifiant unique du processus (0 à size-1)
int MPI_Finalize(void); // Ferme l'environnement MPI
```

3. Communications Point-à-Point

- **Bloquantes** : `MPI\_Send` et `MPI\_Recv`. La fonction ne rend la main que lorsque le tampon d'envoi peut être réutilisé ou que le message est reçu. Risque élevé de **Deadlock (blocage mutuel)** si l'ordre des envois/réceptions est mal conçu.
- **Non-bloquantes** : `MPI\_Isend` et `MPI\_Irecv`. Elles initient l'opération et retournent immédiatement, permettant de superposer le calcul et la communication. L'utilisateur doit vérifier la complétion avec `MPI\_Wait` ou `MPI\_Test`.

4. Communications Collectives

Elles impliquent impérativement **tous** les processus du communicateur (ex: `MPI\_COMM\_WORLD`).

- **MPI_Bcast** : Le processus racine (root) diffuse la même donnée à tous les autres processus.
- **MPI_Scatter** : La racine découpe un tableau en morceaux égaux et envoie un morceau à chaque processus.
- **MPI_Gather** : L'inverse de Scatter. La racine rassemble les morceaux provenant de tous les processus pour reconstruire le tableau complet.
- **MPI_Reduce** : Collecte les données de tous les processus et applique une opération de réduction mathématique (ex : `MPI\_SUM`, `MPI\_MAX`, `MPI\_MIN`) pour stocker le résultat final uniquement sur la racine.
- **MPI_Allreduce** : Équivalent à un `MPI\_Reduce` suivi d'un `MPI\_Bcast` : tout le monde obtient le résultat final.

Exemple de Code C / MPI Complet

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size, value = 0, global_sum = 0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank * 2; // Chaque processus génère une valeur locale

    // Réduction globale de la somme de toutes les valeurs sur le processus 0
    MPI_Reduce(&value, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Le processus 0 a collecté la somme totale : %d\n", global_sum);
    }

    MPI_Finalize();
    return 0;
}
```

Chapitre 5 : Conception de Programmes Parallèles (Méthodologie PCAM)

1. Les Niveaux de Parallélisme

Le parallélisme s'applique à plusieurs échelles : au niveau des instructions (ILP - géré par le processeur), au niveau des boucles/threads (géré par le programmeur ou compilateur), ou au niveau des tâches/jobs (géré par l'OS ou le gestionnaire de batch).

2. La Méthodologie PCAM de Ian Foster

1. **Partitioning (Partitionnement)** : Découper le problème en petits morceaux de calculs et de données autonomes.
 - *Décomposition de domaine* : On découpe les données (ex: diviser une matrice ou une image entre les processeurs).
 - *Décomposition fonctionnelle* : On découpe les actions ou les fonctions à réaliser (ex: une tâche lit les fichiers, une autre calcule, une troisième écrit).
2. **Communication** : Déterminer la structure des échanges d'informations nécessaires entre les sous-tâches pour gérer les dépendances.
3. **Agglomeration (Agglomération)** : Regrouper les petites tâches en structures plus grandes pour s'adapter au matériel cible et réduire le coût global des communications.
4. **Mapping (Affectation)** : Assigner chaque tâche agglomérée à un processeur ou cœur physique spécifique de manière optimisée.

3. Granularité du Calcul et Équilibrage de Charge (Load Balancing)

- **Granularité Fine** : Quantité de calcul faible entre chaque communication. Avantage : équilibrage de charge facile. Inconvénient : coût de communication prohibitif.
- **Granularité Forte (Coarse-grained)** : Grandes phases de calcul isolées avec peu de communications. Avantage : excellente efficacité. Inconvénient : parallélisme plus dur à équilibrer.
- **Load Balancing** : Éviter qu'un processeur reste inactif pendant qu'un autre travaille. La vitesse globale du programme parallèle est toujours dictée par le processeur **le plus lent**. Utilisation fréquente de bassins de tâches (task pools) dynamiques si la charge est imprévisible.

Chapitre 6 : Métrologie, Mesures de Performance et Benchmarks

1. Métriques Fondamentales

- **Speedup (S_p)** : Rapport entre le temps d'exécution séquentiel et le temps sur p processeurs.

$$S_p = T_1 / T_p$$

- **Efficacité (E_p)** : Mesure le rendement de l'utilisation des processeurs.

$$E_p = S_p / p$$

Une efficacité idéale vaut 1 (100%).

2. Les Lois Limites du Parallélisme

- **Loi d'Amdahl (Strong Scaling)** : Modélise le speedup maximal pour une taille de problème **fixe**, où f désigne la fraction strictement séquentielle du code :

$$S_p = 1 / (f + (1 - f) / p)$$

Limite fondamentale : Quand $p \rightarrow \infty$, le speedup maximal est borné par $1 / f$.

- **Loi de Gustafson (Weak Scaling)** : Modélise le speedup lorsque la charge globale augmente proportionnellement avec le nombre de processeurs (le temps d'exécution reste stable) :

$$S_p = p + (1 - p) \times f$$

3. Modèle Roofline

Modèle d'évaluation de la performance liant les GFLOPS à l'Intensité Arithmétique (FLOPS/Octet). Il définit deux zones limites :

- **Memory-Bound** : Performance limitée par la bande passante mémoire ($\text{ext}\{GFLOPS\} = \text{ext}\{Intensité\} \times \text{ext}\{Bande Passante\}$).
- **Compute-Bound** : Performance plafonnée par la puissance de calcul crête théorique du CPU.

Exemple Numérique : Application de la Loi d'Amdahl

Si un programme passe 10% de son temps dans une fonction purement séquentielle (chargement de fichiers, initialisations), alors $f = 0.10$.

Même si l'on déploie une infinité de cœurs parallèles ($p \rightarrow \infty$), le Speedup maximal absolu sera : $S = 1 / 0.10 = 10$. Le programme ne pourra jamais être accéléré plus de 10 fois.

Chapitre 7 : Optimisation de Code Séquentiel et Vectorisation

1. Techniques d'Optimisation des Boucles

- **Loop Unrolling (Déroulage)** : Dupliquer le corps d'une boucle pour réduire le coût d'évaluation de la condition de saut et exposer plus d'instructions parallèles au processeur.
- **Loop Interchange (Interversion)** : Modifier l'ordre d'imbrication des boucles pour correspondre au mode de stockage en mémoire (en C, stockage par lignes "Row-major"). Assure un parcours contigu et maximise les hits de cache.
- **Cache Blocking (Tiling)** : Structurer le calcul sur des sous-blocs de données de taille réduite afin qu'ils tiennent intégralement dans le cache L1/L2, évitant ainsi les rechargements depuis la RAM.

Exemple d'Interversion de Boucle en C

Non optimisé (parcours par colonnes, sauts mémoire constants) :

```
for(int j=0; j<N; j++)
    for(int i=0; i<N; i++)
        matrix[i][j] = 0; // Mauvaise localité spatiale
```

Optimisé (parcours par lignes, mémoire contiguë) :

```
for(int i=0; i<N; i++)
    for(int j=0; j<N; j++)
        matrix[i][j] = 0; // Excellente localité spatiale (Cache Hits ++)
```

2. Obstacles majeurs à l'optimisation automatique du compilateur

- Les dépendances de données complexes au sein des itérations d'une boucle.
- Les appels répétitifs à des fonctions ou la présence de structures conditionnelles complexes (`if-else` imprévisibles) au cœur des boucles internes.
- Les opérations d'entrées/sorties (I/O, comme `printf` ou l'écriture sur disque) au milieu des calculs intensifs.

Chapitre 8 : OpenMP - Parallélisation en Mémoire Partagée

1. Modèle de Programmation

OpenMP fonctionne sur des systèmes multi-cœurs à **mémoire partagée**. Tous les threads s'exécutent en même temps et ont accès au même espace d'adressage RAM. La parallélisation se fait via des directives de compilation insérées dans le code C.

2. Concepts Critiques et Concurrency

- **Race Condition (Situation de compétition)** : Erreur grave se produisant lorsque plusieurs threads tentent de modifier simultanément une même variable partagée sans synchronisation appropriée. Le résultat final devient alors imprévisible.
- **Section Critique** : Bloc de code dont l'accès est strictement restreint à un unique thread à la fois pour préserver l'intégrité des données.

3. Directives et Clauses Majeures

- **#pragma omp parallel for** : Crée une équipe de threads et répartit automatiquement les itérations de la boucle `for` sous-jacente parmi ces threads.
- **private(var)** : Chaque thread possède sa propre copie locale non initialisée de la variable. Sa valeur initiale est perdue à l'entrée de la zone.
- **firstprivate(var)** : Identique à `private`, mais la copie locale de chaque thread est automatiquement initialisée avec la valeur que possédait la variable juste avant la zone parallèle.
- **shared(var)** : Une seule et unique instance de la variable existe en RAM, accessible en lecture/écriture par tous les threads.
- **reduction(opérateur:variable)** : Crée une variable privée par thread pour accumuler localement (ex: faire une somme), puis combine automatiquement et de manière sécurisée toutes les valeurs locales à la sortie de la boucle.

Exemple de Code OpenMP : Calcul de Produit Scalaire

```
#include <omp.h>
#include <stdio.h>

int main() {
    int N = 1000;
    double A[1000], B[1000], dot_product = 0.0;

    // Initialisation bidon
    for(int i=0; i<N; i++) { A[i] = 1.0; B[i] = 2.0; }

    #pragma omp parallel for reduction(+:dot_product) num_threads(4)
    for (int i = 0; i < N; i++) {
        dot_product += A[i] * B[i];
    }

    printf("Produit scalaire total = %f\n", dot_product);
    return 0;
}
```

Chapitre 9 : Programmation GPU avec CUDA

1. Modèle Hôte-Accélérateur (Heterogeneous Computing)

Le calcul s'effectue de manière hybride : l'**Hôte (CPU)** gère le flux de contrôle séquentiel complexe, et le **Device (GPU)** prend en charge les calculs massivement parallèles grâce à ses milliers de cœurs d'exécution simplifiés.

2. Hiérarchie des Threads en CUDA

L'exécution d'un code parallèle (appelé un **Kernel**) s'appuie sur une hiérarchie stricte :

- Une **Grille (Grid)** contient un ensemble de Blocs de threads.
- Un **Bloc (Block)** contient un ensemble de Threads (limité généralement à 1024 threads par bloc).
- Variables internes d'indexation : ``threadIdx.x``, ``blockIdx.x``, ``blockDim.x``, ``gridDim.x``.

Calcul de l'index global unique (1D) : $int\ idx = threadIdx.x + blockIdx.x * blockDim.x;$

3. Gestion de la Mémoire et cycle d'exécution

1. Allocation de la mémoire sur le GPU via la fonction ``cudaMalloc()``.
2. Transfert des données d'entrée du CPU vers le GPU via ``cudaMemcpy(..., cudaMemcpyHostToDevice)``.
3. Lancement du Kernel avec la syntaxe de configuration d'exécution : ``nom_kernel<<<nombre_de_blocs, threads_par_bloc>>>(...)``.
4. Récupération des résultats du GPU vers le CPU via ``cudaMemcpy(..., cudaMemcpyDeviceToHost)``.
5. Libération des espaces mémoires alloués sur le GPU avec ``cudaFree()``.

4. Mots-clés de Fonctions CUDA

Mot-clé	Exécuté sur	Appelé depuis	Notes
<code>__global__</code>	Device (GPU)	Host (CPU)	Définit obligatoirement un Kernel. Doit impérativement renvoyer <code>`void`</code> .
<code>__device__</code>	Device (GPU)	Device (GPU)	Fonction utilitaire interne pour le GPU.
<code>__host__</code>	Host (CPU)	Host (CPU)	Fonction CPU standard classique (comportement par défaut).

Exemple de Code CUDA Complet : Addition de Tableaux

```

#include <cuda_runtime.h>
#include <stdio.h>

// Le Kernel CUDA s'exécutant sur le GPU
__global__ void addArrays(float *A, float *B, float *C, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}

int main() {
    int N = 50000;
    size_t size = N * sizeof(float);

    // Allocation des pointeurs Hôte (CPU)
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);
    float *h_C = (float*)malloc(size);

    // Initialisation des données CPU
    for(int i=0; i<N; i++) { h_A[i] = 1.0f; h_B[i] = 2.0f; }

    // Allocation Mémoire sur le Device (GPU)
    float *d_A, *d_B, *d_C;
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // Transfert CPU -> GPU
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Configuration de l'exécution : 256 threads par bloc
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    // Lancement asynchrone du kernel sur le GPU
    addArrays<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

    // Attente de la fin de l'exécution du GPU
    cudaDeviceSynchronize();

    // Transfert GPU -> CPU
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    printf("Vérification de l'index 0 : %f + %f = %f\n", h_A[0], h_B[0], h_C[0]);

    // Libération des ressources GPU
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);
    return 0;
}

```